

Disassembly: objdump, Ghidra, IDA

Understand the binary mouth

```
file ./app ; readelf -h ./app; readelf -s ./app | head
nm ./app | less          # symbol table
```

ELF (Linux) / PE (Windows). Sections: .text (code), .data/.rodata (data), .bss (uninit). Symbols = function names (they make life easy; stripping removes them, and RE becomes slower).

objdump basics

```
objdump -d ./app | head -50      # disassembly of .text
objdump -M intel -d ./app | grep main -A 20
objdump -t ./app | grep main     # find main's address
```

Key mnemonics on x86-64

```
mov rax, 7      # load immediate
lea rdi, [rip+foo] # effective address (string table)
call 0x4012d0   # function call
test edi, edi   # flags for / jnz (if (a==0))
cmp rax, 0x2a   # compare
jnz/jz/jg      # jumps
push rbp ; mov rbp, rsp # frame prologue
```

Ghidra vs IDA

IDA Pro (paid) – the king of decompilers; auto-structure

Ghidra (free, NSA) – decompiles, code samples, scripting in Python

For learning: Ghidra wins (free + great)

For Windows RE: x64dbg (debugger) pairs with Ghidra

The beginner loop

1. file + readelf (what is it?)
2. strings (what does it echo/check?) – fast win
3. objdump -d main (see the flow)
4. Ghidra import + auto-analysis + decompile main
5. spot the check (cmp/jne) and its constant

Caution zone

RET the security concern: this is *your* learning lab. Analyzing malware in a VM with samples from controlled CTF/labs is the professional path; never random email attachments.

RE is a magnifying glass, not a dark spell. Strings + objdump + Ghidra solve 80% of the 'what does this binary do' question.

[← Module start](#)

[Debugging with GDB: Breakpoint...](#)